RELATIONAL-DATABASE-ANALYSIS

Author: Rohit Kumar Malik

Table of Contents

Question 1: Relational algebra 2

Question 2: SQL – SELECT queries 6

Question 3: Further SQL 12

Question 4: Normalisation 15

Question 5: Conceptual Design 20

Question 1: Relational algebra

1.

$\Pi_{AthleteName,CountryName}(ATHLETE)$

This query retrieves the names and countries of all athletes. The projection operator (Π) is applied to the ATHLETE table to focus on the attributes AthleteName and CountryName. By utilizing projection, extraneous columns are removed, resulting in essential information about each athlete in the database.

2.

$\Pi_{EventName,ScheduledStart}(\sigma_{VenueName='Velodrome'}(EVENT))$

This query identifies events scheduled to take place at the Velodrome. The selection operator (σ) filters the EVENT relation where the venue is 'Velodrome.' The projection operator (Π) then extracts the event name and scheduled start time, providing relevant details solely for this venue.

3.

$\Pi_{AthleteName}(\sigma_{Location='RiodeJaneiro' \vee Location='SaoPaolo'}(ATHLETE \bowtie FINAL \bowtie VENUE))$

This query finds the names of athletes who participated in events held in either Rio de Janeiro or Sao Paulo. The selection operator (σ) filters venues located in these two cities. The VENUE and FINAL tables are then combined to link events with their respective venues. Following this, the results are combined with the ATHLETE table to identify the competing athletes. Finally, the projection operator (Π) displays the names of athletes in these locations.

4.

$\Pi_{AthleteName}(\sigma_{CountryName='Brazil' \wedge Medal='Gold'}(ATHLETE \bowtie FINAL))$

To identify Brazilian athletes who won gold medals, the ATHLETE and FINAL tables are combined. This relationship links each athlete to their event results. The selection operator (σ) is used to filter Brazilian athletes who have won gold medals in their respective sports. The projection operator (Π) displays only the names of the Brazilian gold athletes who won gold medals.

5.

$\Pi_{EventName,ScheduledStart,VenueName,Location}(\sigma_{Capacity>50000}(Event \bowtie Venue))$

This query provides entire event details for large venues, with a capacity of more than 50,000. The selection operator (σ) filters venues depending on capacity. A join between the EVENT and VENUE tables ties each event to its associated location. The projection operator (Π) provides precise information on the events, including venue parameters.

6.

$\Pi_{VenueName,Location,EventName}((VENUE) \bowtie_{left} EVENT)$

This query provides a list of all venues, including their location and any upcoming events. A left outer join between the VENUE and EVENT databases guarantees that venues without scheduled events remain visible, with NULL values for the event fields. The projection operator (Π) simply displays the venue name, location, and event data, including active and unused venues.

7.

$\Pi_{AthleteName}(\sigma_{CountryName='Australia' \wedge Medal='Gold' \wedge VenueName='AquaticsStadium'}((ATHLETE \bowtie FINAL) \bowtie EVENT))$

This query identifies Australian athletes who won gold medals in events held at the Aquatics Stadium. The ATHLETE and FINAL tables are initially combined to connect athletes and their event results. The findings are then combined with the EVENT table to retrieve venue data. The selection operator (σ) is used to filter Australian athletes who won gold medals in the Aquatics Stadium. The projection operator (Π) obtains the names of the athletes, focusing on Australian gold medalists at the venue.

8.

$r1 \leftarrow \pi_{AthleteNo}(\sigma_{EventName='Men\ 100m' \wedge Place=1}(Final))$

$r2 \leftarrow \pi_{AthleteNo}(\sigma_{EventName='Men\ 200m' \wedge Place=1}(Final))$

$\pi_{AthleteName}(Athlete \bowtie (r1 \cap r2))$

This query returns athletes who finished first in both the Men's 100m and 200m events. The selection and projection operators in r1 are used to retrieve the athlete numbers of the first-place winners in the Men's 100m. The same operation is then applied to the Men's 200m event, resulting in r2. The intersection (r1 ∩ r2) ensures that only competitors who finished first in both events are included. Finally, a natural join with the ATHLETE table retrieves and projects the athletes' names, revealing those who excelled in both sprints.

9.

$\Pi_{AthleteName}(\sigma_{EventName='Womens´1500mFreestyle' \wedge Medal=NULL}(ATHLETE \bowtie FINAL))$

This query is for athletes who did not win any medals in the Women's 1500m Freestyle event. A natural join between the ATHLETE and FINAL tables connects athletes with their event participation. The selection operator (σ) selects individuals who competed in the Women's 1500m Freestyle event but did not obtain a medal, as indicated by Medal = NULL. The projection operator (Π) takes just the names of athletes who did not get medals in the event, ensuring a comprehensive list.

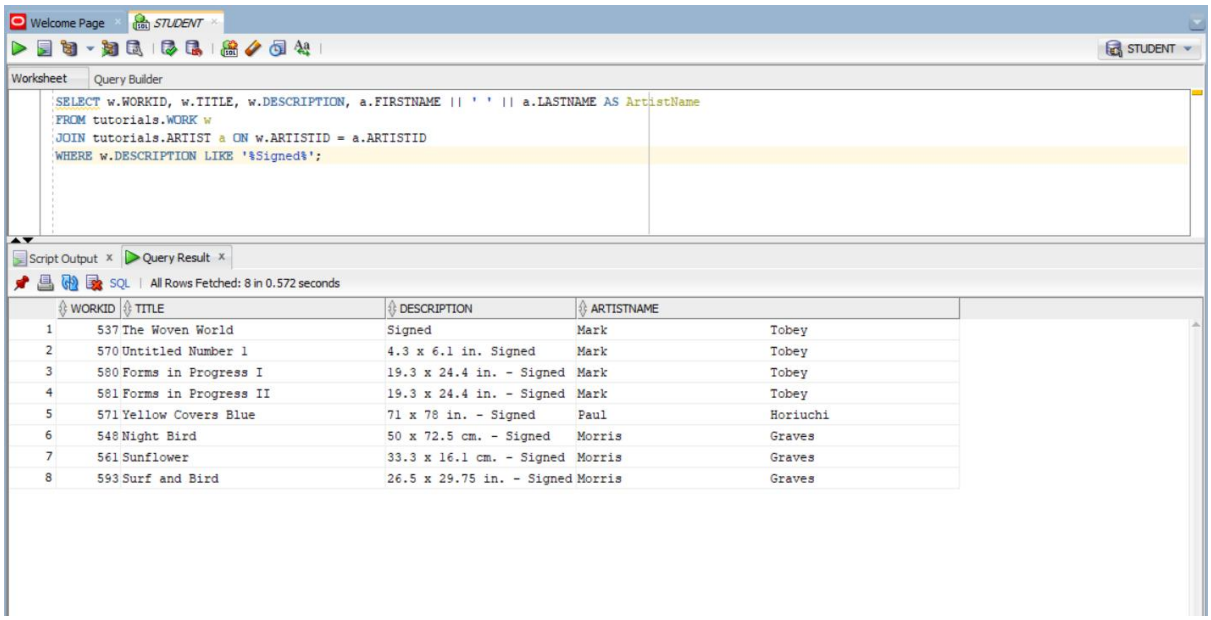
10.

$\Pi_{AthleteName,AthleteNo,EventName}(\sigma_{Medal='Gold'}(ATHLETE \bowtie FINAL)) \div \Pi_{AthleteNo,EventName}(FINAL)$

This query returns athletes who won gold medals in every event they competed in. A natural join between the ATHLETE and FINAL tables is done, confining results to athletes who earned a gold medal with $\sigma_{Medal='Gold'}$. The division operator (÷) is used to ensure these athletes won gold in all events they entered. This operation ensures that the set of events in which the athlete earned a gold medal corresponds to the set of events in which they competed, recognizing competitors with a perfect record. The projection operator (Π) selects athlete names and related events.

Question 2: SQL – SELECT queries

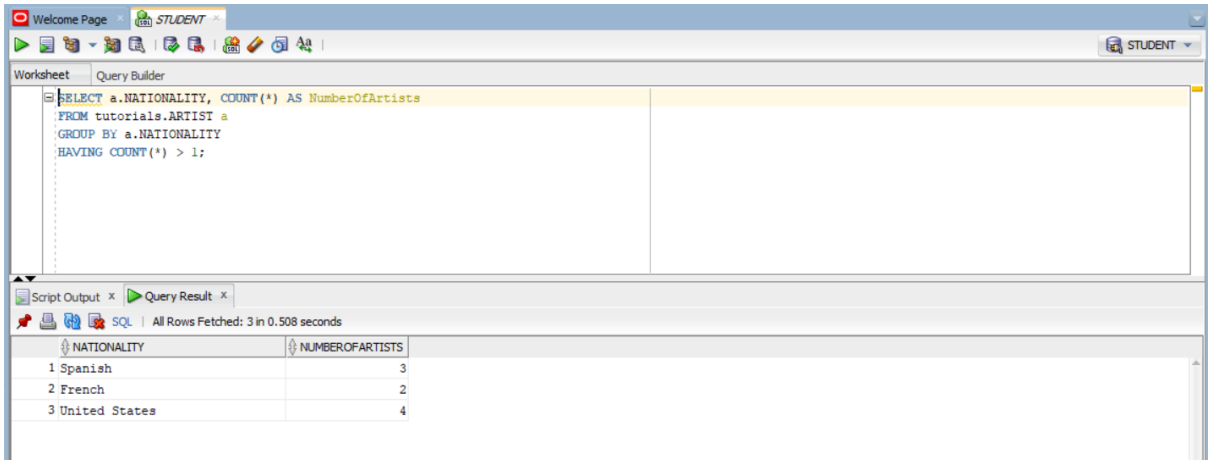
1.



	WORKID	TITLE	DESCRIPTION	ARTISTNAME	
1	537	The Woven World	Signed	Mark	Tobey
2	570	Untitled Number 1	4.3 x 6.1 in. Signed	Mark	Tobey
3	580	Forms in Progress I	19.3 x 24.4 in. - Signed	Mark	Tobey
4	581	Forms in Progress II	19.3 x 24.4 in. - Signed	Mark	Tobey
5	571	Yellow Covers Blue	71 x 78 in. - Signed	Paul	Horiuchi
6	548	Night Bird	50 x 72.5 cm. - Signed	Morris	Graves
7	561	Sunflower	33.3 x 16.1 cm. - Signed	Morris	Graves
8	593	Surf and Bird	26.5 x 29.75 in. - Signed	Morris	Graves

This query retrieves all works of art with the word 'Signed' in the description. The query begins by joining the WORK and ARTIST tables and matching the artist to each work of art using the ARTISTID. Then, a WHERE clause is used to filter for works with the term 'Signed' in the description, ensuring that only relevant results are returned. The SELECT clause is used to display the title, artist's full name, and description.

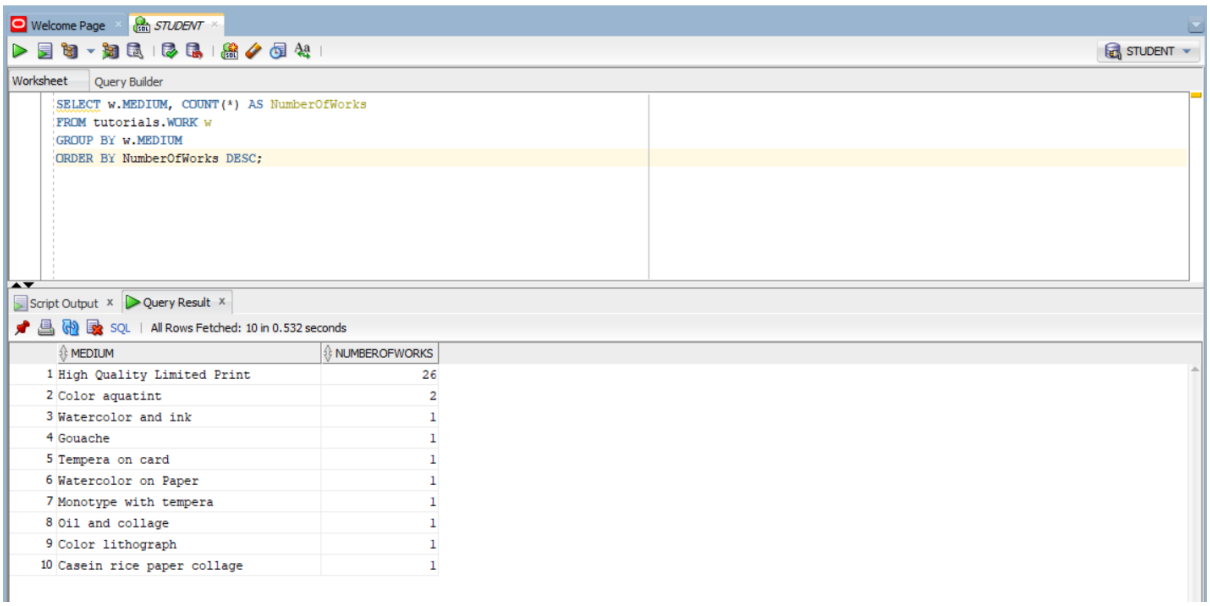
2.



	NATIONALITY	NUMBEROFARTISTS
1	Spanish	3
2	French	2
3	United States	4

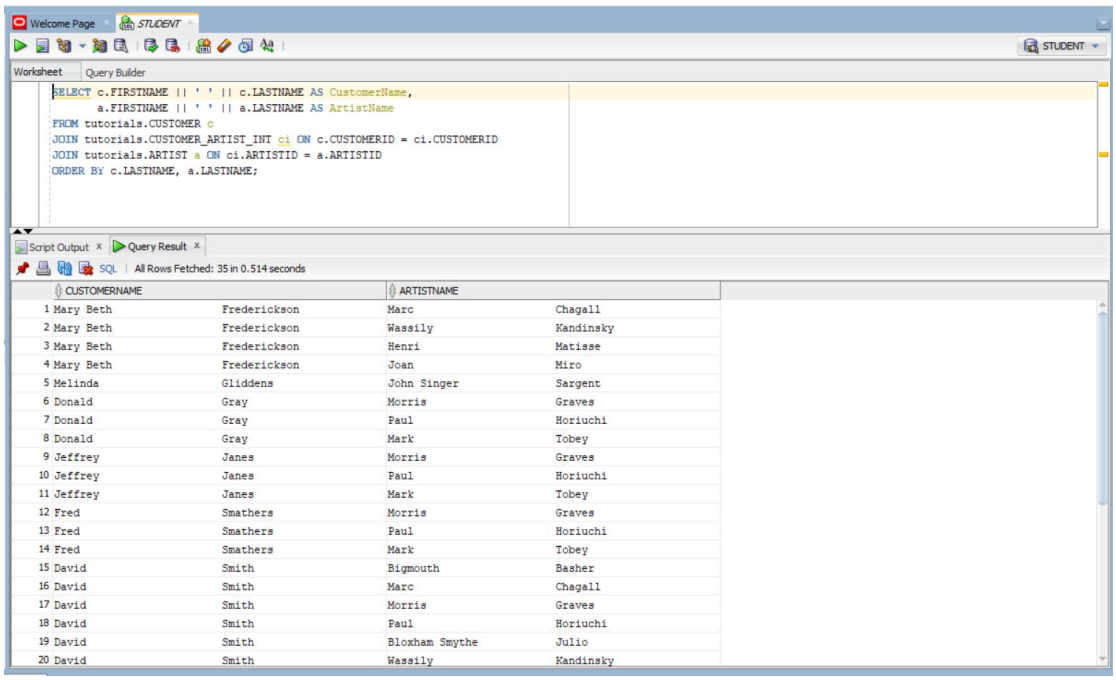
This query locates nationalities with more than one artist represented in the database. It groups the artists into categories based on their nationality and counts how many belong to each group. The query then uses a HAVING clause to display only nationalities where the number of artists exceeds one. This helps to identify nationalities that are well-represented in the gallery.

3.



This query counts the number of works dependent on the media (e.g., oil, watercolor). The total number of works for each medium is calculated by grouping the results by the MEDIUM field in the WORK table. The results are then sorted in descending order, with the medium with the most works appearing first. This reveals which mediums are most common in the gallery.

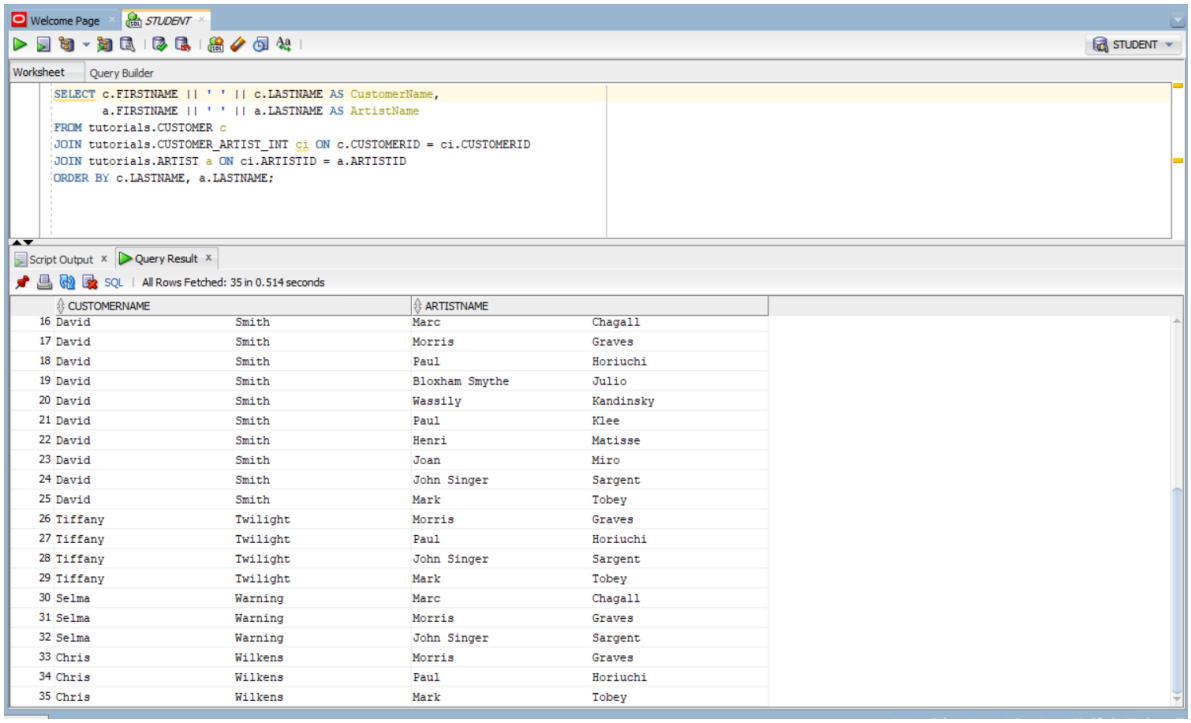
4.



```
SELECT c.FIRSTNAME || ' ' || c.LASTNAME AS CustomerName,
       a.FIRSTNAME || ' ' || a.LASTNAME AS ArtistName
FROM tutorials.CUSTOMER c
JOIN tutorials.CUSTOMER_ARTIST_INT ci ON c.CUSTOMERID = ci.CUSTOMERID
JOIN tutorials.ARTIST a ON ci.ARTISTID = a.ARTISTID
ORDER BY c.LASTNAME, a.LASTNAME;
```

All Rows Fetched: 35 in 0.514 seconds

	CUSTOMERNAME		ARTISTNAME	
1	Mary Beth	Frederickson	Marc	Chagall
2	Mary Beth	Frederickson	Wassily	Kandinsky
3	Mary Beth	Frederickson	Henri	Matisse
4	Mary Beth	Frederickson	Joan	Miro
5	Melinda	Gliddens	John Singer	Sargent
6	Donald	Gray	Morris	Graves
7	Donald	Gray	Paul	Horiuchi
8	Donald	Gray	Mark	Tobey
9	Jeffrey	Janes	Morris	Graves
10	Jeffrey	Janes	Paul	Horiuchi
11	Jeffrey	Janes	Mark	Tobey
12	Fred	Smathers	Morris	Graves
13	Fred	Smathers	Paul	Horiuchi
14	Fred	Smathers	Mark	Tobey
15	David	Smith	Bigmouth	Basher
16	David	Smith	Marc	Chagall
17	David	Smith	Morris	Graves
18	David	Smith	Paul	Horiuchi
19	David	Smith	Bloxham Smythe	Julio
20	David	Smith	Wassily	Kandinsky



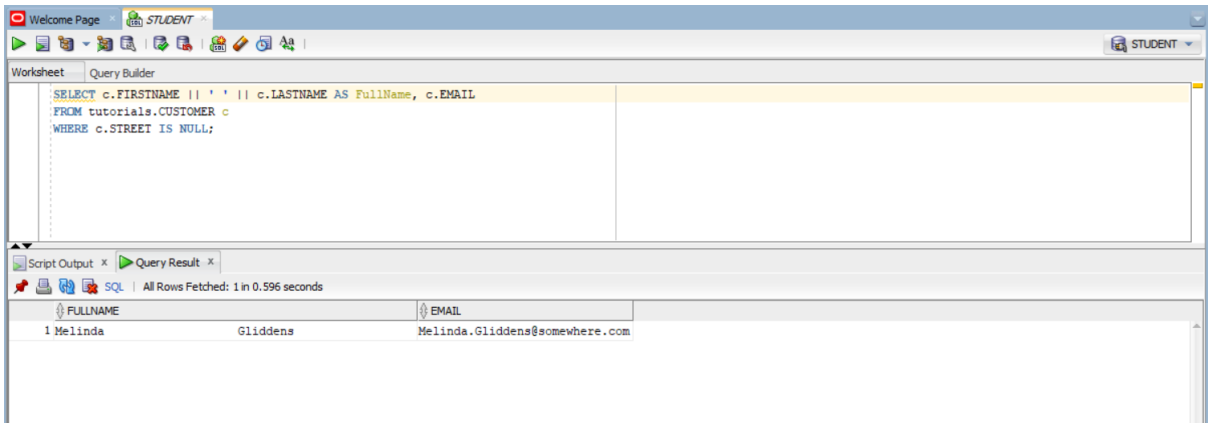
```
SELECT c.FIRSTNAME || ' ' || c.LASTNAME AS CustomerName,
       a.FIRSTNAME || ' ' || a.LASTNAME AS ArtistName
FROM tutorials.CUSTOMER c
JOIN tutorials.CUSTOMER_ARTIST_INT ci ON c.CUSTOMERID = ci.CUSTOMERID
JOIN tutorials.ARTIST a ON ci.ARTISTID = a.ARTISTID
ORDER BY c.LASTNAME, a.LASTNAME;
```

All Rows Fetched: 35 in 0.514 seconds

	CUSTOMERNAME		ARTISTNAME	
16	David	Smith	Marc	Chagall
17	David	Smith	Morris	Graves
18	David	Smith	Paul	Horiuchi
19	David	Smith	Bloxham Smythe	Julio
20	David	Smith	Wassily	Kandinsky
21	David	Smith	Paul	Klee
22	David	Smith	Henri	Matisse
23	David	Smith	Joan	Miro
24	David	Smith	John Singer	Sargent
25	David	Smith	Mark	Tobey
26	Tiffany	Twilight	Morris	Graves
27	Tiffany	Twilight	Paul	Horiuchi
28	Tiffany	Twilight	John Singer	Sargent
29	Tiffany	Twilight	Mark	Tobey
30	Selma	Warning	Marc	Chagall
31	Selma	Warning	Morris	Graves
32	Selma	Warning	John Singer	Sargent
33	Chris	Wilkens	Morris	Graves
34	Chris	Wilkens	Paul	Horiuchi
35	Chris	Wilkens	Mark	Tobey

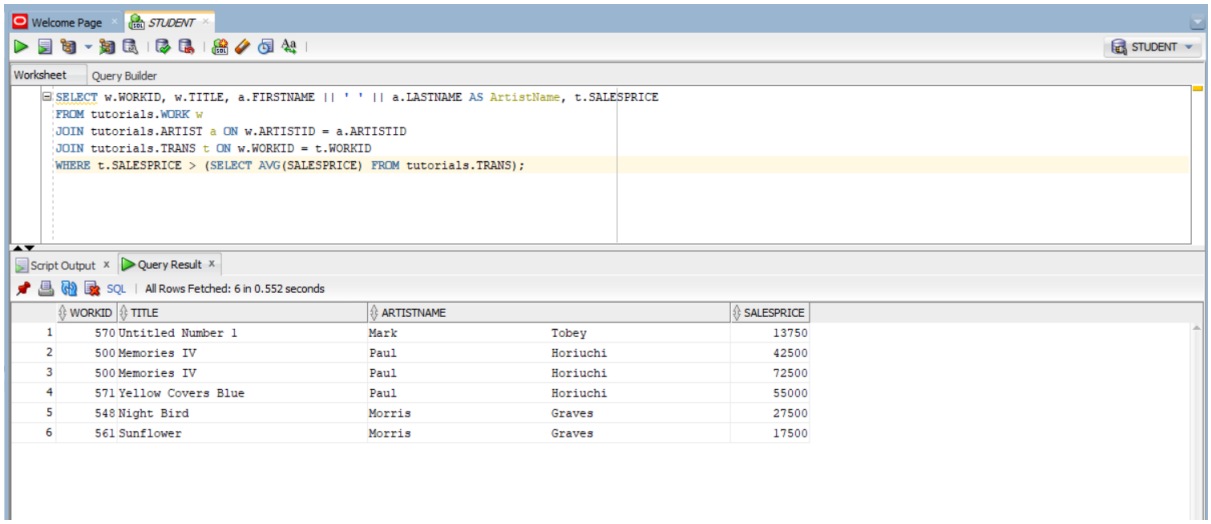
This query returns a list of customers and the artists they are interested in. It begins by joining the CUSTOMER, CUSTOMER_ARTIST_INT, and ARTIST tables. The query sorts the results alphabetically, first by the customer's last name, then by the artist's last name. This way, the data is nicely arranged and easy to follow, revealing the connections between clients and their preferred artists.

5.



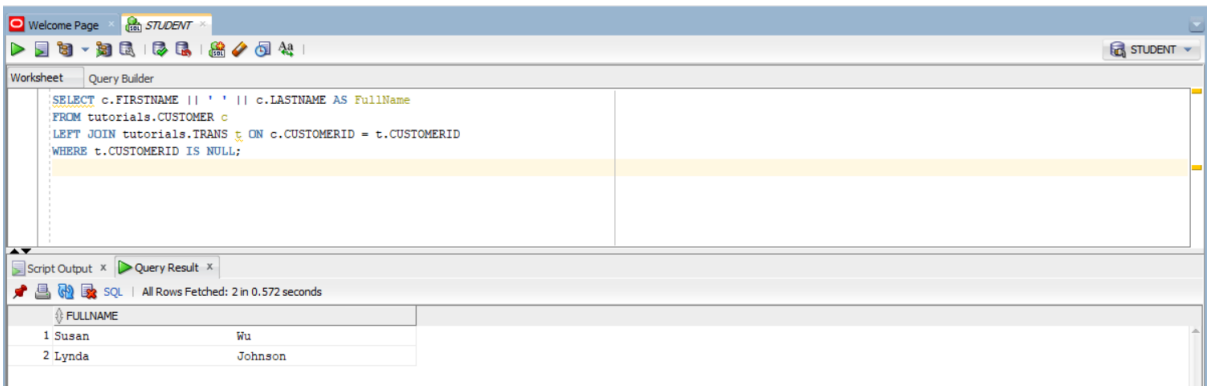
This query focuses on customers who are missing address details, specifically the STREET column. The query uses a WHERE clause to detect clients with null values in their address fields and then chooses their full name and email address. This is useful for finding incomplete records in the database that require attention or revisions.

6.



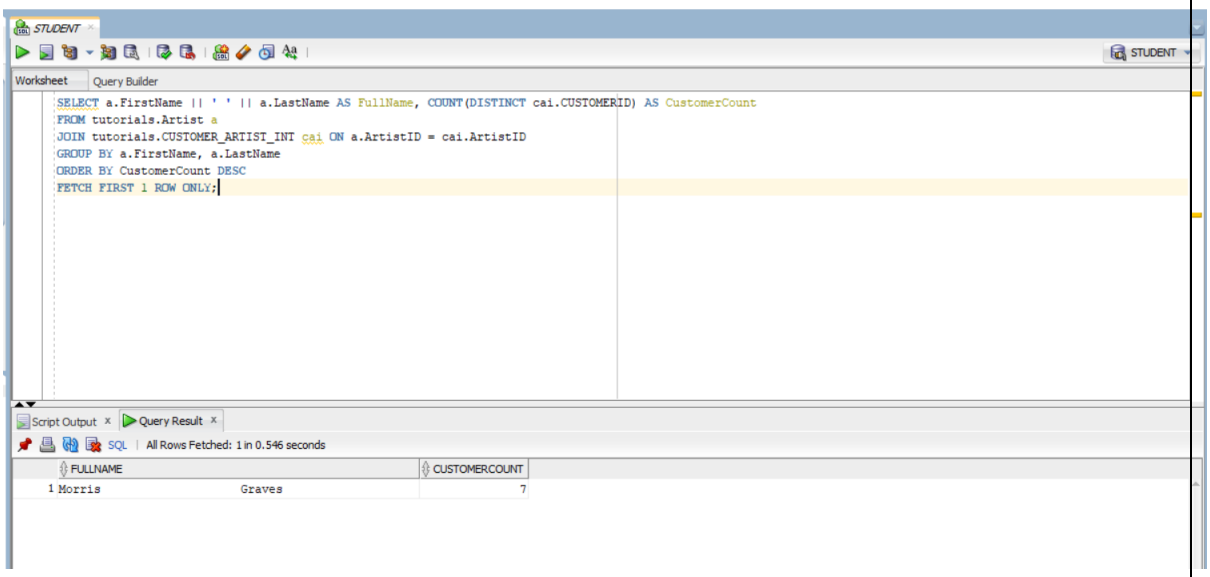
This query returns works of art that sold for more than the average sales price. It begins by calculating the average sales price using a subquery, after which it filters the TRANS table results to only include those where the sale price is above the average. The WORK and ARTIST tables are also combined to contain the title and artist's name for each work of art, ensuring that the final output is comprehensive and instructive.

7.



This query returns a list of customers who have never purchased a work of art. It does this by choosing customers who do not have any matching entries in the TRANS table, followed by a LEFT JOIN and a WHERE clause that filters for null values in the TRANS table's WORKID field. This method ensures that only clients without connected transactions are returned, giving a clear picture of who has yet to make a purchase.

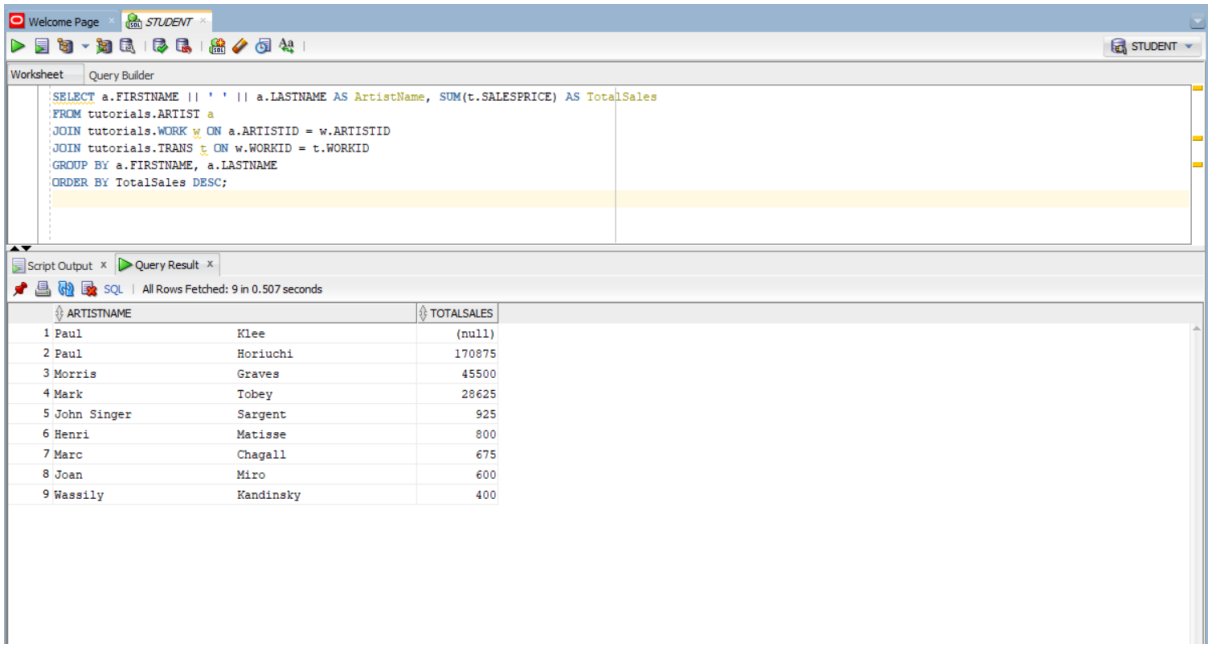
8.



This query determines the artist with the most client interest. The query uses the CUSTOMER_ARTIST_INT and ARTIST columns to count the number of consumers interested in each artist. The results are then categorized by artist and sorted by count in descending order. This assures that the artist with the

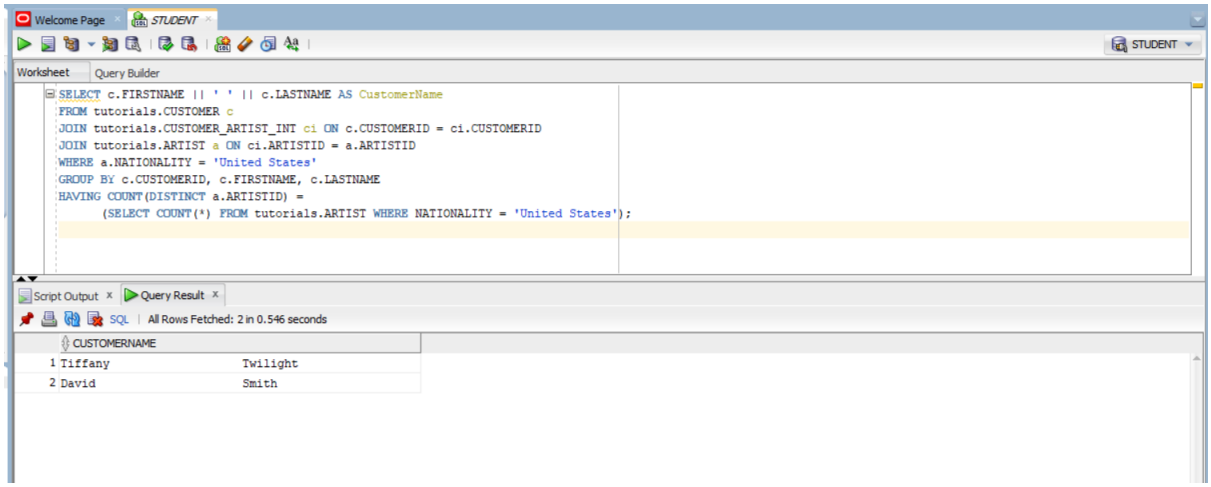
most interested clients appears at the top, along with the exact number of customers.

9.



This query calculates each artist's total sales by adding the sale prices of their works from the TRANS table. The piece and ARTIST tables are joined to display the artist's full name for each work. The results are then organized by artist and sorted by total sales amount in descending order. This clearly shows which artists have been the most successful in terms of sales, from highest to lowest.

10.

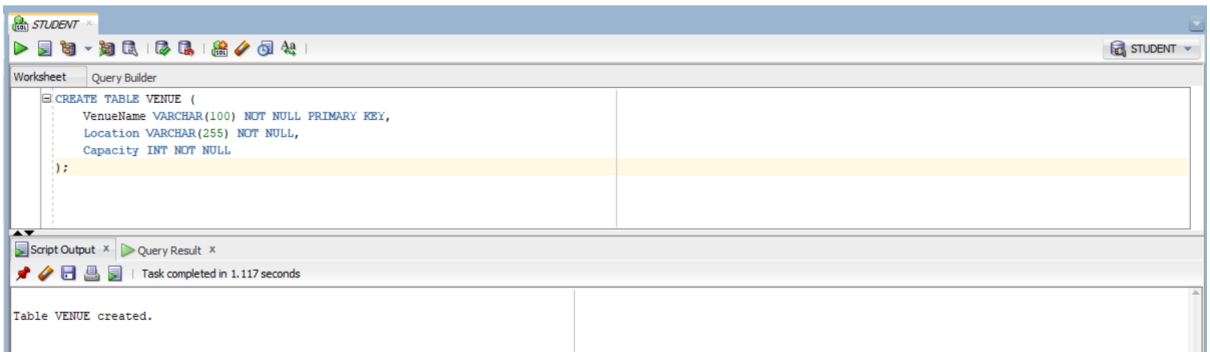


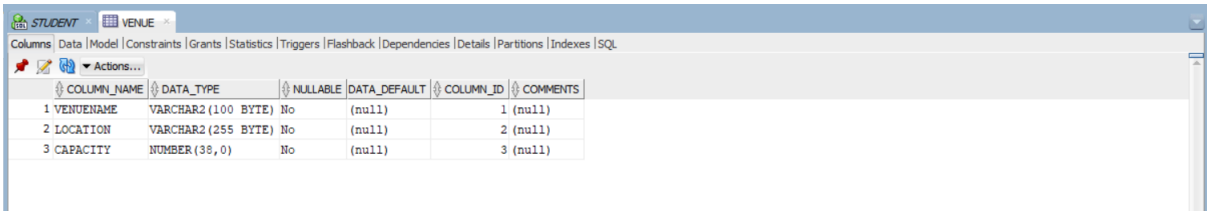
This query discovers customers who are interested in all artists from the United States. The query joins the CUSTOMER, CUSTOMER_ARTIST_INT, and ARTIST tables and then filters for artists from the United States. The HAVING condition ensures that only customers who have expressed an interest in each US-based artist are included in the results. This query effectively highlights the customers, with an emphasis on American artists.

Question 3: Further SQL

1.

```
CREATE TABLE VENUE (
VenueName VARCHAR(100) NOT NULL PRIMARY KEY,
Location VARCHAR(255) NOT NULL,
Capacity INT NOT NULL
);
```

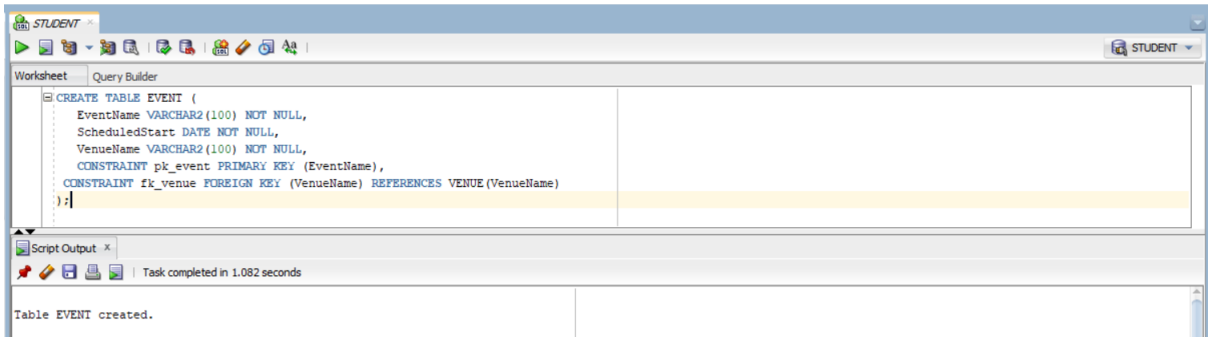


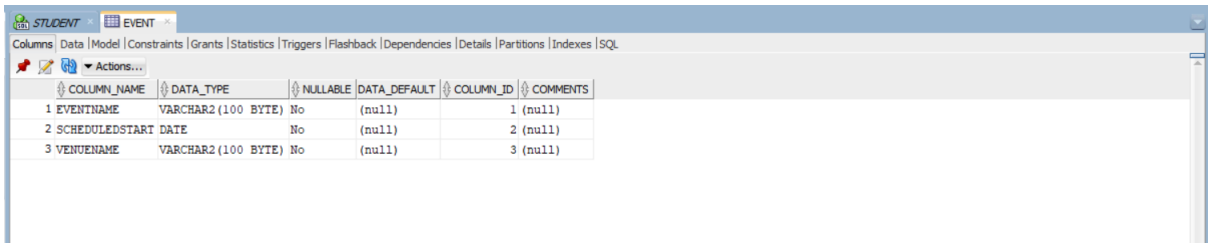


This SQL command creates the VENUE table, which contains three columns: VenueName, Location, and Capacity. The data types are chosen to reflect the nature of each attribute, guaranteeing that no values are null. The VenueName column is selected as the primary key, guaranteeing that each venue's name is unique and cannot be replicated.

2.

```
CREATE TABLE EVENT (
EventName VARCHAR2(100) NOT NULL,
ScheduledStart DATE NOT NULL,
VenueName VARCHAR2(100) NOT NULL,
CONSTRAINT pk_event PRIMARY KEY (EventName),
CONSTRAINT fk_venue FOREIGN KEY (VenueName) REFERENCES
VENUE(VenueName)
);
```

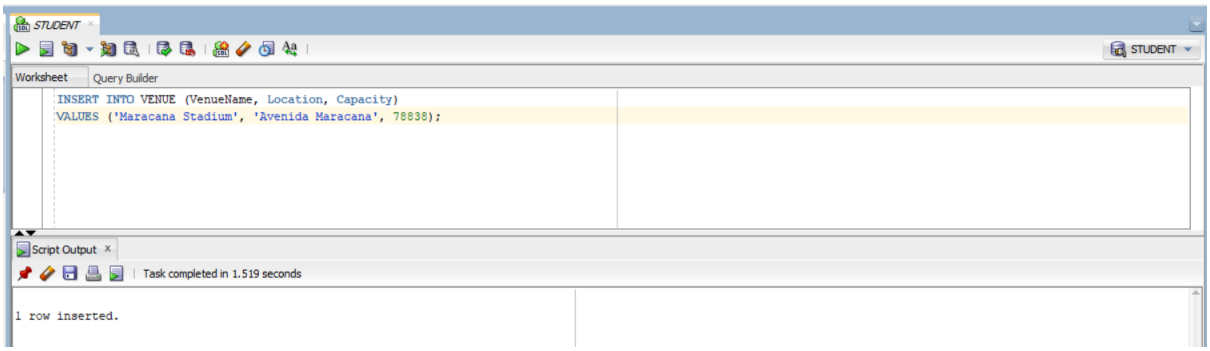




This SQL command creates the EVENT table containing columns for EventName, ScheduledStart, and VenueName. EventName is the primary key, and a foreign key constraint connects VenueName to the VENUE table, ensuring referential integrity. This restriction ensures that a venue cannot be deleted while events are still scheduled at that location.

3.

```
INSERT INTO VENUE (VenueName, Location, Capacity)
VALUES ('Maracana Stadium', 'Avenida Maracana', 78838);
```

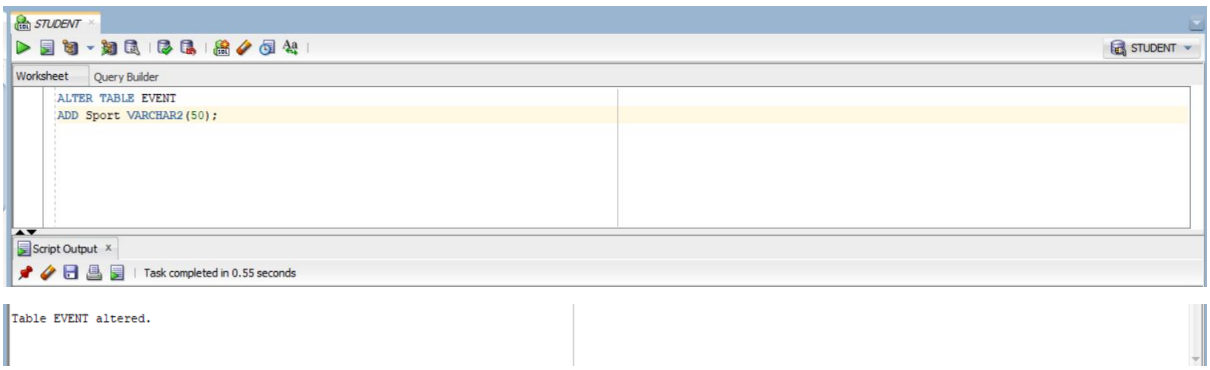


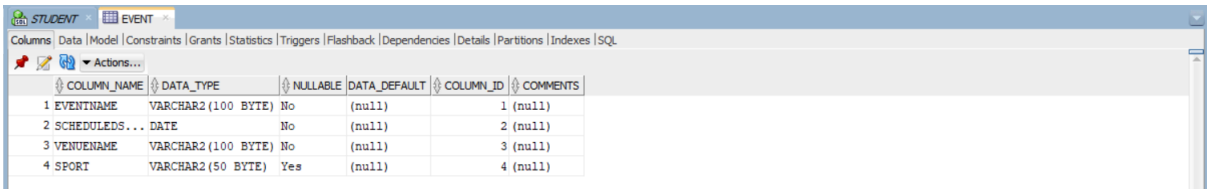
This SQL statement creates a new record in the VENUE database, including the Maracana Stadium's location and capacity. The INSERT INTO command ensures that the stadium's information is accurately entered into the database.

4.

ALTER TABLE EVENT

ADD Sport VARCHAR2(50);



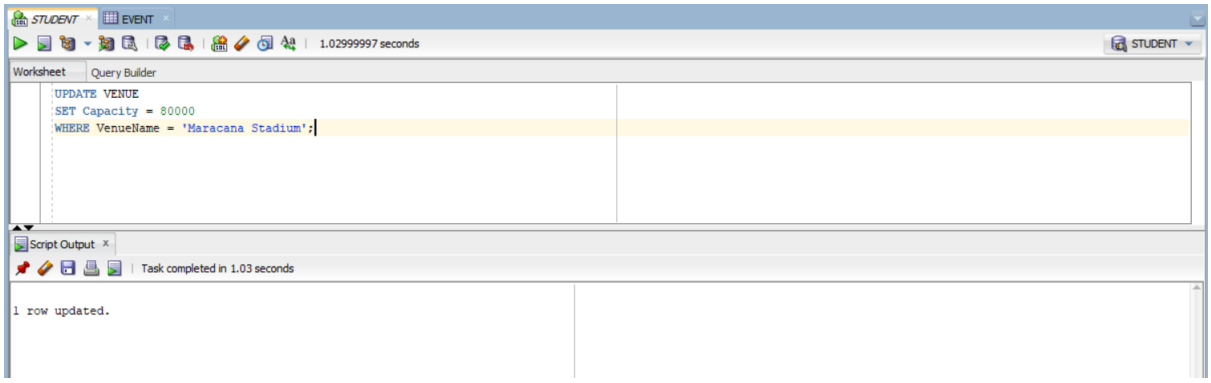


	COLUMN_NAME	DATA_TYPE	NULLABLE	DATA_DEFAULT	COLUMN_ID	COMMENTS
1	EVENTNAME	VARCHAR2(100 BYTE)	No	(null)	1	(null)
2	SCHEDULEDS...	DATE	No	(null)	2	(null)
3	VENUENAME	VARCHAR2(100 BYTE)	No	(null)	3	(null)
4	SPORT	VARCHAR2(50 BYTE)	Yes	(null)	4	(null)

This SQL command alters the EVENT table by adding a new column, Sport, that contains information on the type of sport associated with each event. The data type chosen enables the specification of numerous sports, such as Athletics or Tennis, which increases the table's adaptability.

5.

```
UPDATE VENUE
SET Capacity = 80000
WHERE VenueName = 'Maracana Stadium';
```



This SQL query updates Maracana Stadium's capacity to reflect the new number of 80,000. The UPDATE command modifies the Capacity column for the specified venue, ensuring that the database contains up-to-date information.

Question 4: Normalisation

Report on Normalization of the Car-Service Relation:

1.

Identifying Problems with the Current Design:

The current design of the Car-Service relation in the Automotive Mechanic and Repair business database has various flaws that come from a lack of normalization. These issues cause inefficiencies, data redundancies, and inconsistencies in the database, making it difficult to manage and query.

Absence of a Primary Key: The current design does not provide a unique identifier for each transaction. Without a primary key, distinguishing between distinct service records becomes difficult, posing challenges for tracking and modifying specific transactions. This absence affects the data's integrity,

particularly when many services are recorded for the same customer or vehicle.

Data Redundancy: Customer information, such as CustName and CustDOB, appears on numerous service records. This repetition not only takes up unnecessary storage space but also increases the possibility of contradictions. For example, if a customer's details change (such as a name change), the information must be updated in multiple places, increasing the possibility of errors.

Inconsistent Data Formats: Some fields, such as ServiceFee, have inconsistent data entry, with some entries as numbers (e.g., 199) and others as words (e.g., "Fifty"). Furthermore, incorrect data inputs, such as "99999" in StaffSpeciality, show data entry issues. This discrepancy impedes data retrieval, analysis, and validation.

Partial Dependencies: Certain non-key properties, such as CustName and CustDOB, rely on only a portion of the composite key, which violates normalization rules. These fields should be based on a unique customer identification (e.g., CustID) rather than a collection of unrelated data like ServiceDate or CarRego.

Transitive Dependencies: The present relationship also has transitive dependencies. StaffName and StaffSpeciality fields are included in the same table as the service details, despite the fact that they are dependent on the Staff ID. This layout creates duplication, making it more difficult to update staff information uniformly across many entries.

- **Update, Insertion, and Deletion Anomalies:**

    - **Update Anomalies:** Modifying customer or vehicle information requires modifications to many records. Failure to update every

instance leads to inconsistent data within the database.

- **Insertion Anomalies:** It is not feasible to add a new customer or car without first submitting a service record. This design issue means that the system cannot maintain customer or vehicle information in the absence of a service transaction.

- **Deletion Anomalies:** Deleting a service transaction may mistakenly erase critical customer or vehicle information that should remain in the system independent of service records.

2.

Proposed Normalized Design:

To address these issues, the Car-Service relation must be restructured using database normalization principles up to the Third Normal Form (3NF). The new design divides the huge relation into smaller, linked tables, reducing redundancy and anomalies while improving system accuracy and maintainability.

Customer Table:

CustID	CustName	CustDOB
1	Lois Price	08-09-1998
2	Justin Thyme	10-10-2000
3	Stan Dupp	10-10-2000
4	Jack Pott	29-03-1981
5	Ina Minat	03-01-1991
6	Mike Rafone	30-01-2004

The Customer table introduces CustID as a primary key to ensure that each customer has a unique identification. By doing so, the system reduces the

need to repeat client information across different service records, making updates easier and enhancing data integrity.

Car Table:

CarID	CarRego	CarMade	CarModel
1	1y568	Toyota	Camry
2	1y576	Honda	CRV
3	1t567	Toyota	Camry
4	1t158	Ford	Focus
5	1y458	Tesla	Model X
6	1i132	Holden	Astra
7	1l884	BMW	Z4
8	1i123	Black car	Astra

The Car table uses CarID as a primary key, eliminating the need to manually enter car information for each service. This reduces repetition and ensures that each car is uniquely recognized throughout the database.

Staff Table:

StaffID	StaffName	StaffSpeciality
1	Ali	Service tech
2	Chip	Auto body
3	Holly	Service tech

The Staff table uses StaffID as the primary key to separate staff-related data from service details. This structure guarantees that employee information is consistently updated, avoiding conflicts across numerous entries.

Service Table:

ServiceID	CustID	CarID	ServiceDate	ServiceFee	StaffID
1	1	1	19-05-2023	199	1
2	2	2	08-04-2022	400	2
3	3	3	09-05-2023	199	1
4	4	4	02-10-2022	50	3
5	1	5	01-08-2022	199	1
6	5	6	15-09-2022	190	2
7	6	7	15-09-2022	120	1
8	5	8	10-08-2019	269	1

The Service table holds service-specific information and ensures that each transaction is uniquely recognized by ServiceID. The table connects to the CustID, CarID, and StaffID to standardize data entry and eliminate unnecessary information. The ServiceFee values are now uniformly formatted as numbers, which resolves previous inconsistencies.

Justification for Design Changes:

Primary Key Integration: Each table now has a unique primary key, ensuring that all records are distinct and recognizable, hence improving data integrity.

Elimination of Redundancy: Separating customer, automobile, and staff information into separate databases reduces duplicate data entry, simplifies changes, and lowers the possibility of discrepancies.

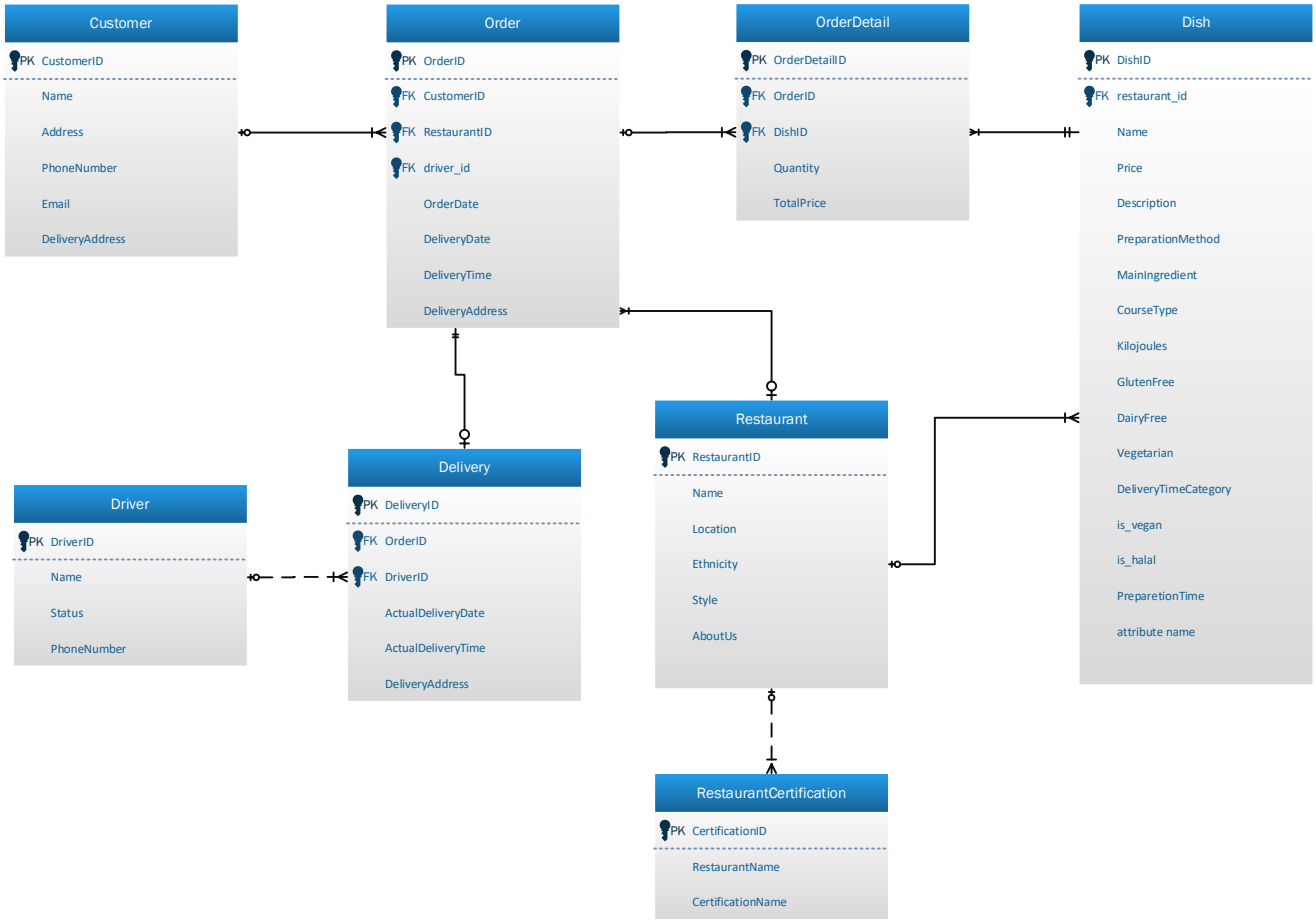
Consistent Data Formats: Setting the ServiceFee field to numeric values assures consistency, which improves query accuracy and performance.

Adherence to 3NF: The new design follows the rules of the Third Normal Form (3NF), which eliminates both partial and transitive dependencies. This assures that non-key attributes only rely on their primary keys, making the database more efficient.

Anomaly prevention: The new structure addresses update, insertion, and deletion issues by allowing customer, automobile, and staff information to exist independently of service records.

Question 5: Conceptual Design

1. ERD:



2.

Assumptions for FineFoods4U ERD:

<u>Customer Registration and Address Validity:</u>

Assumption: Customers must register using legitimate contact information (name, address, phone number, email) and a delivery address. Only

addresses in the same suburb as the restaurant are permitted to complete orders.

Explanation: This assumption assures that delivery is possible within the restaurant's geographic boundaries, as defined by Bill's business model.

Single delivery per order:

Assumption: A customer may only order from one restaurant for each delivery.

Explanation: Each order is assigned to a single restaurant, in accordance with Bill's decision to limit deliveries to a single restaurant per transaction, ensuring quality and fast delivery.

Driver's Assignment:

Assumption: Each order is allocated to a single driver, who can manage one delivery at a time.

Explanation: The "one-to-one" relationship between the Order and Delivery entities assumes that a driver handles a single order from pickup to delivery, providing accountability and proper tracking.

OrderDetails Capture Dish Information:

Assumption: The OrderDetail entity records the quantity and total price of each dish ordered. Each OrderDetail corresponds to exactly one dish.

Explanation: This simplifies the process of itemizing an order and computing the final price while also allowing many dishes to be associated with the same order.

Restaurant certifications:

Assumption: The RestaurantCertification entity can record numerous certifications for restaurants, including vegan, halal, and gluten-free.

Explanation: Certifications are crucial for customers with unique dietary needs, and each certification is linked to the appropriate restaurant to accommodate variable query criteria.

Dish Preparation and Delivery Category:

Assumption: Dishes have specified cooking methods (e.g., fried, steamed) and are classified by delivery time (quick, regular, worth the wait).

Explanation: This data is maintained in the Dish entity to present clients with information that may impact their decisions, allowing the system to filter based on preparation and delivery constraints.

Driver Status Tracking:

Assumption: The Driver entity stores driver status to assess their availability for deliveries.

Explanation: Tracking driver status ensures that the system knows which drivers are currently available for delivery assignments, allowing orders to be assigned more efficiently.

Dish Details, Including Nutritional Information:

Assumption: Each dish provides specific information, such as kilojoules, dietary alternatives (gluten-free, dairy-free, vegetarian), and other characteristics such as halal and vegan.

Explanation: This data contributes to meeting customer expectations for healthier and more personalized food selections, as well as supporting queries about these dietary preferences.

Order and Delivery Time Recording:

Assumption: The actual delivery time and date are saved in the Delivery entity, allowing Bill to monitor his performance in terms of timely delivery.

Explanation: This enables Bill to measure and report on delivery efficiency, as well as check punctuality and service quality.

Restaurant and Dish Descriptions:

Assumption: Restaurants and foods include full details (such as restaurant information and dish preparation methods).

Explanation: This allows customers to make informed decisions and is

consistent with Bill's objective to provide complete menus for both web-based and in-house promotions.